

Azure Cloud MLOps

End-to-End Spam Detection Pipeline

Note: Code implementations have been omitted from this version.

Focus is placed strictly on architectural flows, cloud portal steps, and operational concepts.

Overview

This document is a comprehensive hands-on guide covering the end-to-end Machine Learning Operations (MLOps) lifecycle on Microsoft Azure and Render.

We will build a production-ready AI Spam Detector from scratch, progressing through seven distinct phases:

1. Data Collection & Storage: Provisioning Azure Blob Storage and uploading datasets.
 2. Data Preparation & EDA: Setting up an Azure ML Workspace, creating compute resources, registering datasets, and performing cleaning.
 3. Model Training & MLflow Tracking: Training a Multinomial Naive Bayes model and tracking metrics dynamically in Azure using MLflow.
 4. Evaluation & Tuning: Reviewing the workspace dashboard, running hyperparameter sweeps, and running AutoML.
 5. Model Deployment: Deploying the registered model to a Managed Online Endpoint.
 6. Web App Deployment: Wrapping the model into a FastAPI app with a glassmorphic HTML UI, and deploying it on Render via GitHub.
 7. Monitoring & Retraining: Tracking drift alerts, production logs, and designing auto-retraining architectures.
-

Module 1 - Data Collection & Storage

In our cloud-native machine learning lifecycle, all files, models, and run outputs are managed on Azure. The first stage is

Data Collection & Storage. We use Azure Blob Storage (similar to AWS S3) as our secure data repository.

We will set up the storage account, create containers for our datasets, and upload our raw spam detection dataset (spam.csv).

Step 1: Create a Storage Account (Azure Portal)

1. Open the Azure Portal.
2. In the top search bar, type Storage accounts and select it.
3. Click the + Create button in the top left.
4. Fill in the basics tab:
 - Subscription: Select your subscription.
 - Resource Group: Click Create new and name it spam-detection-rg.
 - Storage account name: Enter a unique lowercase name (e.g., spamdetectionsa2026).
 - Region: Select a region (e.g., East US).
 - Performance: Standard.
 - Preferred storage type: Select Azure Blob Storage or Azure Data Lake Storage (the recommended choice for machine learning datasets and models).
 - Redundancy: Select Locally-redundant storage (LRS) (recommended for lab, learning, and dev/test environments as it is the lowest-cost option).
 - All Other Settings (Advanced, Networking, Data protection, Encryption, Tags): You can safely leave these at their default settings!
5. Click Review + create at the bottom, then click Create.
6. Wait for deployment, and click Go to resource.

Step 2: Create Containers for Raw & Processed Data

1. Inside your Storage Account page, scroll down the left menu to Data storage and click Containers.
2. Click + Container at the top.
 - Name: raw-data
 - Public access level: Private
3. Click + Container again.
 - Name: processed-data
 - Public access level: Private
- Click Create.

Step 3: Upload the Raw Dataset

1. Click on the raw-data container in the list.
2. Click Upload at the top.
3. Browse and select your local dataset file: spam.csv.
4. Click Upload.

Optional: Programmatic Ingestion (Python Upload)

If you want to upload the file programmatically from a script, run this Python script:

Module 2 - Data Preparation & EDA

Before training a model, we must explore and clean the dataset. In Azure, you can manage and track datasets securely using Datastores (connections to storage services) and Data Assets (versioned data references) inside Azure Machine Learning Studio.

Step 1: Create an Azure Machine Learning Workspace

If you don't have an Azure ML Workspace yet:

1. Search for Azure Machine Learning in the Azure Portal.
2. Click + Create -> New workspace.
3. In the Basics tab, configure the following:
 - Subscription: Select your subscription.
 - Resource Group: Choose the one you created in Step 1 (e.g., spam-detection-rg).
 - Name: Enter spam-detection-mlw.
 - Region: Choose the same region as your storage account (e.g., East US).
 - Storage account, Key vault, Application insights: Leave these at their default Create new settings. Azure will automatically generate its own default storage account to save workspace system logs and internal run artifacts.
(Note: Keep the "Create new" default here; do not select the storage account you created in Module 1. We will link your Module 1 storage account as a Datastore in Step 2 below).
 - Container registry: Keep it at None (Azure will automatically create one later when we build custom environments/containers).
4. Click Review + create at the bottom, then click Create.
5. Once deployed, click Go to resource, then click Launch studio (which takes you to ml.azure.com).

Step 2: Register a Datastore

A Datastore securely saves connection details to your Azure Blob Storage so you don't have to hardcode connection strings in your scripts.

1. In Azure ML Studio, click Data in the left menu.
2. Select the Datastores tab at the top, then click + Create.
3. Configure the Datastore:
 - Datastore name: spam_storage_ds
 - Datastore type: Azure Blob Storage
 - Account selection method: From Azure subscription.
 - Storage account: Choose the storage account you created in Step 1 (e.g., spamdetectionsa2026).
 - Blob container: Select raw-data (which has your spam.csv).
 - Authentication type: Select Account key.
 - Account key: Paste the access key of your Storage Account (copy this from your Storage Account page -> Access keys -> key1 Key in the Azure Portal).
4. Click Create.

Step 3: Register a Data Asset

1. In Azure ML Studio, go to Data on the left menu, select the Data assets tab, and click + Create.
2. Set Name: spam_raw_data, Type: File (uri_file). Click Next.
3. Choose From Azure storage -> Select the Datastore spam_storage_ds -> Browse to spam.csv. Click Next.
4. Review the schema details and click Create.

Step 4: Create a Compute Instance to Run Notebooks

We will create a single Compute Instance that serves as our development environment for the entire MLOps workflow.

1. Go to Compute in the left menu of Azure ML Studio.
2. Under the Compute instances tab, click + New.

3. Configure the following settings:

- Compute name: Enter a unique name (e.g., spam-detection-instance).
- Virtual machine type: Select CPU.
- Virtual machine size: Select Standard_DS11_v2 (2 cores, 14GB RAM, 28GB storage).

4. Click Review + create to review your settings. Verify they match these recommended defaults:

- Scheduling: Auto shutdown should be Enabled (set to shut down after 60 minutes of inactivity to prevent extra charges).

5. Click Create (takes ~3 minutes).

6. Once running, go to Notebooks in the left menu, click + -> Create new file, select Notebook (.ipynb), and attach it to your running compute instance. Choose the preinstalled kernel: Python 3.10 - SDK v2.

Step 5: Data Preparation Notebook Cells

Write this Python code in the cells of your Jupyter Notebook to clean the dataset:

Module 3 - Model Training & MLflow Tracking

We will train our Multinomial Naive Bayes model directly in our Jupyter Notebook running on our compute instance. All training execution, tracking, and model registering will happen inside the notebook cells using MLflow integrated into Azure ML.

Cell 1: Load Clean Dataset & Split Data

Cell 2: Train & Evaluate the Model (Tracked via MLflow)

We Vectorize the text using TF-IDF and train a Naive Bayes classifier. We wrap the training loop with MLflow so that parameters and metrics are automatically tracked inside our Azure ML Workspace history!

Cell 3: Register the Model in the Workspace Registry

Registering the model stores it in the central registry so it can be versioned and deployed as an endpoint.

Module 4 - Evaluation & Tuning

Once your training cells have run, you can track and evaluate model performance metrics, compare runs, and optimize hyperparameters.

Part A - Metric Tracking & Evaluation (Azure ML Studio UI)

Any metrics logged in your training script via `mlflow.log_metric()` are saved automatically in your workspace history.

1. Open Azure ML Studio.
2. On the left sidebar, click Jobs.
3. Select your experiment name (e.g., spam-detection-training).
4. Click on your training run (named with a random adjective-noun string).

5. Click on the Metrics tab to view logged charts (accuracy, custom loss, F1-scores).
6. To compare multiple runs side-by-side: Go back to the experiment runs list, check the boxes next to different runs, and click Compare at the top.

Part B - Hyperparameter Tuning (Simple Notebook Sweep)

We can perform a hyperparameter sweep directly inside our notebook using a Python loop and MLflow:

Part C - Automated ML (AutoML via UI)

You can run Automated ML (AutoML) directly from the UI without writing code:

1. In Azure ML Studio, click Automated ML on the left menu.
2. Click + New Automated ML job.
3. Choose your dataset: Select spam_raw_data. Click Next.
4. Configure Job:
 - Task type: Classification.
 - Target column: Select label.
 - Compute type: Select Serverless (this runs on temporary serverless VMs managed by Azure).
5. Click Submit to run.

Module 5 - Real-time Model Deployment

In this stage, we will deploy our registered model to an Azure ML Managed Online Endpoint (a hosted web service API) directly from our Jupyter Notebook.

Cell 1: Write the Scoring Script (score.py)

We use the Jupyter `%%writefile` magic command to write our prediction scoring script (score.py) directly from the notebook onto the Compute Instance storage.

Cell 2: Define and Create the Online Endpoint

Cell 3: Deploy the Model to the Endpoint

This provisions a virtual machine, packages our model and scoring script in a Docker container, and hosts it.

Troubleshooting Common Deployment Failures

- **SubscriptionNotRegistered Error:** If endpoint creation fails with this error, go to the Azure Portal, open your Subscription, click Resource Providers on the left menu, search for Microsoft.Cdn and Microsoft.PolicyInsights, and click Register.
- **Failed Deployment (blue) in Unrecoverable State:** If a previous deployment execution failed, that slot remains in an unrecoverable state. Modify `name="blue"` in the notebook to `name="green"` or `name="blue-v2"` and re-run.
- **OutOfQuota Error:** If you have VM quota limitations on standard clouds, deploy locally on Docker inside the compute instance:

Module 6 - Web App Deployment on Render

Now that your model is hosted as an Azure ML Online Endpoint, we will build a FastAPI web application to serve as a production middleware API and frontend interface. Finally, we will containerize this application and deploy it to Render directly from GitHub.

Step 1: Project Directory Structure

First, create the project directory and file structure on your local machine using your operating system's terminal:

On macOS / Linux:

On Windows (Command Prompt):

This creates the following file hierarchy:

Step 2: Configure Local Environment Variables

Rather than typing shell export commands, we use a .env file to load settings automatically during local development.

1. Open Azure ML Studio -> Endpoints -> Select your endpoint -> Go to the Consume tab. Copy REST endpoint and Primary key.
2. Open .env and paste:
3. Open .gitignore and add:

Step 3: Create the FastAPI Backend (main.py)

Write the FastAPI code to serve the frontend, ingest incoming requests, and proxy them to the Azure ML online endpoint securely.

Step 4: Create the UI Frontend (templates/index.html)

The frontend is a modern glassmorphic interface that communicates with our FastAPI proxy server. It uses CSS gradients, layout styling, dynamic alerts, and loading symbols.

Step 5: Configure Dependencies & Dockerfile

requirements.txt

Dockerfile

Step 6: Local Virtual Environment Setup

1. Create and activate a virtual environment:
 - macOS/Linux: `python3 -m venv venv && source venv/bin/activate`
 - Windows: `python -m venv venv && venv\Scripts\activate.bat`
2. Install dependencies:
3. Run the server locally:

Step 7: Push to GitHub & Deploy to Render

1. Push your code files to a GitHub repository.
 2. Open the Render Dashboard, select New + -> Web Service, and link your repository.
 3. Configure the settings:
 - Runtime: Docker
 - Instance Type: Free
 4. Add the Environment Variables:
 - AZURE_ENDPOINT_URL -> (Your endpoint REST URL)
 - AZURE_API_KEY -> (Your endpoint Primary Key)
 5. Click Deploy.
-

Module 7 - Monitoring & Retraining

Once your online model endpoint is deployed and serving traffic, the final MLOps stage is Monitoring & Retraining.

Part A - Application Insights (System Health & Logs)

Application Insights tracks container execution, endpoint errors, latency, and request metrics.

1. In Azure ML Studio, select your endpoint page, and click the Logs tab to view real-time prints, stdout outputs, and errors from your container's score.py execution.
2. Go to the Details tab and click on the Application Insights resource link to view server latency graphs and load metrics.

Part B - Data Drift Monitoring

Data drift occurs when the input data sent to your live model starts to look significantly different from your baseline training data.

1. Capture inputs: Modify your notebook deployment configuration to enable data collection:
2. Configure a Drift Monitor in Studio: Go to Monitoring on the left menu, select + Create, choose your baseline training dataset (spam_raw_data) and target production folder, set drift metrics, and add email alerts.

Part C - Automated Retraining Architecture

When a drift alert is triggered (e.g., Population Stability Index $PSI > 0.2$), it runs an automated retraining pipeline:

To update the endpoint with a new model version programmatically: